P2372R2
Fixing locale handling in chrono formatters

Published Proposal, 2021-08-06

Authors:

Victor Zverovich

Corentin Jabot

Audience:

LWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Table of Contents

"Mistakes have been made, as all can see and I admit it."

— Ulysses S. Grant

§ 1. The problem

In C++20 "Extending `<chrono>` to Calendars and Time Zones" ([P0355]) and "Text Formatting" ([P0645]) proposals were integrated ([P1361]). Unfortunately during this integration a design issue was missed: `std::format` is locale-independent by default and provides control over locale via format specifiers but the new formatter specializations for chrono types are localized by default and don't provide such control.

For example:

```
std::locale::global(std::locale("ru_RU"));

std::string s1 = std::format("{}", 4.2);         // s1 == "4.2" (not localized)
std::string s2 = std::format("{:L}", 4.2);       // s2 == "4,2" (localized)

using sec = std::chrono::duration<double>;
std::string s3 = std::format("{:%S}", sec(4.2)); // s3 == "04,200" (localized)
```

In addition to being inconsistent with the design of `std::format`, there is no way to avoid locale other than doing formatting of date and time components manually.

Confusingly, some chrono format specifiers such as `%S` may give an impression that they are locale-independent by having a locale's alternative representation like `%OS` while in fact they are not.

The implementation of [P1361] in [FMT] actually did the right thing and made most chrono specifiers locale-independent by default, for example:

```
using sec = std::chrono::duration<double>;
std::string s = fmt::format("{:%S}", sec(4.2));  // s == "04.200" (not localized)
```

This implementation has been available and actively used in this form for 2+ years. The bug in the specification of chrono formatters in the standard and the mismatch with the actual implementation have only been discovered recently and reported in [LWG3547].

§ 2. The solution

We propose fixing this issue by making chrono formatters locale-independent by default and providing the `L` specifier to opt into localized formatting in the same way as it is done for all other standard formatters (format.string.std).

Before	After
`auto s = std::format("{:%S}", sec(4.2));` `// s == "04,200"`	`auto s = std::format("{:%S}", sec(4.2));` `// s == "04.200"`
`auto s = std::format("{:L%S}", sec(4.2));` `// throws format_error`	`auto s = std::format("{:L%S}", sec(4.2));` `// s == "04,200"`

§ 3. Changes from R1

- Replaced two occurrences of "If the *L* option is used, the locale is the locale passed to the formatting function, or otherwise the global locale. If the *L* option is not used, the "C" locale is used." with a paragraph specifying the choice of locale in wording.
- Added the `L` option in `[time.zone.zonedtime.nonmembers]`.

§ 4. Changes from R0

- Kept ostream insertion operators (`operator<<`) locale-dependent for consistency with the rest of the ostream library per LEWG feedback.
- Drive-by fix: made ostream insertion operators of `utc_time`, `tai_time`, `gps_time` and `file_time` use the stream's locale instead of the global locale. They are locale-dependent because the decimal point is localized.
- Drive-by fix: Used the correct locale if the *chrono-specs* is omitted.
- Added LEWG poll results.

§ 5. Locale alternative forms

Some specifiers (`%d %H %I %m %M %S %u %w %y %z`) produce digits which are not localized (aka they use the Arabic numerals `0123456789`) although as we demonstrated earlier `%S` is still using a localized decimal separator. They have an equivalent form (`%Od %OH %OI %Om %OM %OS %Ou %Ow %Oy %Oz`) where the numerals can be localized. For example, Japanese numerals `〇 一 二 三 四 五 ...` can be used as the "alternative representation" by a `ja_JP` locale.

But because the `L` option applies to all specifiers, we do not propose to modify the specifiers.

For example, `"{:L%p%I}"` and `"{:L%p%OI}"` should be valid specifiers producing `午後1` and `午後一` respectively.

Appropriate use of numeral systems for localized numbers and dates requires more work, this paper focuses on a consistent default behavior.

§ 6. The "C" locale

The "C" locale is used in the wording as a way to express locale-independent behavior. The C standard specifies the "C" locale behavior for `strftime` as follows

> In the "C" locale, the E and O modifiers are ignored and the replacement strings for the following specifiers are:
>
> ```
> %a the first three characters of %A.
> %A one of ‘Sunday’, ‘Monday’, ... , ‘Saturday’.
> %b the first three characters of %B.
> %B one of ‘January’, ‘February’, ... , ‘December’.
> %c equivalent to ‘%a %b %e %T %Y’.
> %p one of ‘AM’ or ‘PM’.
> %r equivalent to ‘%I:%M:%S %p’.
> %x equivalent to ‘%y’.
> %X equivalent to %T.
> %Z implementation-defined.
> ```

This makes it possible, as long as the `L` option is not specified, to format dates in environment without locale support (embedded platforms, `constexpr` if someone proposes it, etc).

§ 7. SG16 polls

Poll: LWG3547 raises a valid design defect in [time.format] in C++20.

SF	F	N	A	SA
7	2	2	0	0

Outcome: Strong consensus that this is a design defect.

Poll: The proposed LWG3547 resolution as written should be applied to C++23.

SF	F	N	A	SA
0	4	2	4	1

Outcome: No consensus for the resolution

SA motivation: Migrating things embedded in a string literal is very difficult. There are options to deal with this in an additive way. Needless break in backwards with compatibility.

SG16 recognized that this is a design defect but was concerned about this being a breaking change. However, the following facts were not known at the time of the discussion:

- The implementation of [P1361] in [FMT] is locale-independent. This was the only implementation available for 2+ years and was cited as the only source of implementation experience in the paper.
- Both `%S` and `%OS` depend on locale and there is no locale-independent equivalent.
- The chrono formatting in the Microsoft’s implementation has only been merged into the main branch on 22 April and has bugs that will require breaking changes.
- Some chrono types are partially localized, e.g. `month_day_last{May}` may be formatted as `Mai/last` in a German locale with only month localized.

§ 8. LEWG polls

Poll: Revise D2372 to keep the ostream operators for chrono formatting dependent on the stream locale

SF	F	N	A	SA
10	8	2	0	0

Outcome: Strong Consensus in Favor

Poll: LEWG approves of the direction of this work and encourages further work as directed above with the intent that D2372 (Fixing locale handling in chrono formatters) will land in C++23 and be applied retroactively to C++20

SF	F	N	A	SA
14	8	0	0	0

Outcome: Unanimous approval

§ 9. Implementation experience

The `L` specifier has been implemented for durations in the fmt library ([FMT]). Additionally, some format specifiers like `S` have never used a locale by default so this was a novel behavior accidentally introduced in C++20:

```
std::locale::global(std::locale("ru_RU"));
using sec = std::chrono::duration<double>;
std::string s = fmt::format("{:%S}", sec(4.2)); // s == "04.200" (not localized)
```

This proposed fix has also been implemented and submitted to the Microsoft standard library.

§ 10. Impact on existing code

Changing the semantics of chrono formatters to be consistent with standard format specifiers (format.string.std) is a breaking change. At the time of writing the Microsoft's implementation recenly merged the chrono formatting into the main branch and is known to be not fully conformant. For example:

```
using sec = std::chrono::duration<double>;
std::string s = std::format("{:%S}", sec(4.2)); // s == "04" (incorrect)
```

§ 11. Wording

All wording is relative to the C++ working draft [N4885].

Update the value of the feature-testing macro `__cpp_lib_format` to the date of adoption in [version.syn]:

Change in [time.format]:

chrono-format-spec:
  fill-and-align$_{opt}$ *width*$_{opt}$ *precision*$_{opt}$ *L*$_{opt}$ *chrono-specs*$_{opt}$

A locale used by a formatting function is determined as follows:

- the "C" locale if the `L` option is not present in *chrono-format-spec*, otherwise
- the locale passed to the formatting function if any, otherwise
- the global locale.

2 Each conversion specifier *conversion-spec* is replaced by appropriate characters as described in Table [tab:time.format.spec]; the formats specified in ISO 8601:2004 shall be used where so described. Some of the conversion specifiers depend on ~~the locale that is passed to the formatting function if the latter takes one, or the global locale otherwise.~~ a locale. If the formatted object does not contain the information the conversion specifier refers to, an exception of type `format_error` is thrown.

...

6 If the *chrono-specs* is omitted, the chrono object is formatted as if by streaming it to `std::ostringstream os` with a locale imbued and copying `os.str()` through the output iterator of the context with additional padding and adjustments as specified by the format specifiers.

Change in [time.clock.system.nonmembers]:

```
template<class charT, class traits, class Duration>
  basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const sys_time<Duration>& tp);
```

...

2 *Effects*: Equivalent to:

```
auto const dp = floor(tp);
return os << format(os.getloc(), STATICALLY-WIDEN("{} {}""{:L} {:L}"),
                    year_month_day{dp}, hh_mm_ss{tp-dp});
```

Change in [time.clock.utc.nonmembers]:

```
template<class charT, class traits, class Duration>
  basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const utc_time<Duration>& t);
```

1 *Effects*: Equivalent to:

```
return os << format(os.getloc(), STATICALLY-WIDEN("{:%F %T}""{:L%F %T}"), t);
```

(Adding `os.getloc()` is a drive-by fix.)

Change in [time.clock.tai.nonmembers]:

```
template<class charT, class traits, class Duration>
  basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const tai_time<Duration>& t);
```

1 *Effects*: Equivalent to:

```
return os << format(os.getloc(), STATICALLY-WIDEN("{:%F %T}""{:L%F %T}"), t);
```

Change in [time.clock.gps.nonmembers]:

```
template<class charT, class traits, class Duration>
  basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const gps_time<Duration>& t);
```

1 *Effects*: Equivalent to:

```
return os << format(os.getloc(), STATICALLY-WIDEN("{:%F %T}""{:L%F %T}"), t);
```

Change in [time.clock.file.nonmembers]:

```
template<class charT, class traits, class Duration>
  basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const file_time<Duration>& t);
```

1 *Effects*: Equivalent to:

```
return os << format(os.getloc(), STATICALLY-WIDEN("{:%F %T}""{:L%F %T}"), t);
```

[time.cal.day.nonmembers] is intentionally left unchanged because `%d` is locale-independent.

Change in [time.cal.month.nonmembers]:

```
template<class charT, class traits>
  basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const month& m);
```

7 *Effects*: Equivalent to:

```
return os << (m.ok() ?
  format(os.getloc(), STATICALLY-WIDEN<charT>("{:%b}""{:L%b}"), m) :
  format(os.getloc(), STATICALLY-WIDEN<charT>("{} is not a valid month"),
         static_cast<unsigned>(m)));
```

[time.cal.year.nonmembers] is intentionally left unchanged because `%Y` is locale-independent.

Change in [time.cal.wd.nonmembers]:

```
template<class charT, class traits>
  basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const weekday& wd);
```

6 *Effects*: Equivalent to:

```
return os << (wd.ok() ?
  format(os.getloc(), STATICALLY-WIDEN<charT>("{:%a}""{:L%a}"), wd) :
  format(os.getloc(), STATICALLY-WIDEN<charT>("{} is not a valid weekday"),
         static_cast<unsigned>(wd.wd_)));
```

Change in [time.cal.wdidx.nonmembers]:

```
template<class charT, class traits>
  basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const weekday_indexed& wdi);
```

2 *Effects*: Equivalent to:

```
auto i = wdi.index();
return os << (i >= 1 && i <= 5 ?
  format(os.getloc(), STATICALLY-WIDEN<charT>("{}[{}]""{:L}[{}]"), wdi.weekday(), i) :
  format(os.getloc(), STATICALLY-WIDEN<charT>("{:L}[{} is not a valid index]"),
         wdi.weekday(), i));
```

Change in [time.cal.wdlast.nonmembers]:

```
template<class charT, class traits>
  basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const weekday_last& wdl);
```

2 *Effects*: Equivalent to:

```
return os << format(os.getloc(), STATICALLY-WIDEN<charT>("{:L}[last]"), wdl.weekday());
```

Change in [time.cal.md.nonmembers]:

```
template<class charT, class traits>
  basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const month_day& md);
```

3 *Effects*: Equivalent to:

```
return os << format(os.getloc(), STATICALLY-WIDEN<charT>("{}/{}""{:L}/{:L}"),
                    md.month(), md.day());
```

Change in [time.cal.mdlast]:

```
template<class charT, class traits>
  basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const month_day_last& mdl);
```

9 *Effects*: Equivalent to:

```
return os << format(os.getloc(), STATICALLY-WIDEN<charT>("{}/last""{:L}/last"), mdl.month
```

Change in [time.cal.mwd.nonmembers]:

```
template<class charT, class traits>
  basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const month_weekday& mwd);
```

2 *Effects*: Equivalent to:

```
return os << format(os.getloc(), STATICALLY-WIDEN<charT>("{}/{}""{:L}/{:L}"),
                    mwd.month(), mwd.weekday_indexed());
```

Change in [time.cal.mwdlast.nonmembers]:

```
template<class charT, class traits>
  basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const month_weekday_last& mwdl);
```

2 *Effects*: Equivalent to:

```
return os << format(os.getloc(), STATICALLY-WIDEN<charT>("{}/{}""{:L}/{:L}"),
                    mwdl.month(), mwdl.weekday_last());
```

Change in [time.cal.ym.nonmembers]:

```
template<class charT, class traits>
  basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const year_month& ym);
```

14 *Effects*: Equivalent to:

```
return os << format(os.getloc(), STATICALLY-WIDEN<charT>("{}/{}""{:L}/{:L}"),
                    ym.year(), ym.month());
```

[time.cal.ymd.nonmembers] is intentionally left unchanged because `%F` is locale-independent.

Change in [time.cal.ymdlast.nonmembers]:

```
template<class charT, class traits>
  basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const year_month_day_last& ymdl);
```

12 *Effects*: Equivalent to:

```
return os << format(os.getloc(), STATICALLY-WIDEN<charT>("{}/{}""{:L}/{:L}"),
                    ymdl.year(), ymdl.month_day_last());
```

Change in [time.cal.ymwd.nonmembers]:

```
template<class charT, class traits>
  basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const year_month_weekday& ymwd);
```

11 *Effects*: Equivalent to:

```
return os << format(os.getloc(), STATICALLY-WIDEN<charT>("{}/{}/{}""{:L}/{:L}/{:L}"),
                    ymwd.year(), ymwd.month(), ymwd.weekday_indexed());
```

Change in [time.cal.ymwdlast.nonmembers]:

```
template<class charT, class traits>
  basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const year_month_weekday_last& ymwdl);
```

11 *Effects*: Equivalent to:

```
return os << format(os.getloc(), STATICALLY-WIDEN<charT>("{}/{}/{}""{:L}/{:L}/{:L}"),
                    ymwdl.year(), ymwdl.month(), ymwdl.weekday_last());
```

Change in [time.hms.nonmembers]:

```
template<class charT, class traits, class Duration>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const hh_mm_ss<Duration>& hms);
```

1 *Effects*: Equivalent to:

```
return os << format(os.getloc(), STATICALLY-WIDEN<charT>("{:%T}""{:L%T}"), hms);
```

Change in [time.zone.zonedtime.nonmembers]:

2 *Effects*: Streams the value returned from `t.get_local_time()` to `os` using the format ~~`"%F %T %Z"`~~ `"L%F %T %Z"` .

References

Informative References

[FMT]
Victor Zverovich; et al. The {fmt} library. URL: https://github.com/fmtlib/fmt

[LWG3547]
Corentin Jabot. Time formatters should not be locale sensitive by default. URL: https://cplusplus.github.io/LWG/issue3547

[N4885]
Thomas Köppe; et al. Working Draft, Standard for Programming Language C++. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/n4885.pdf

[P0355]
Howard E. Hinnant; Tomasz Kamiński. Extending to Calendars and Time Zones.. URL: https://wg21.link/p0355

[P0645]
Victor Zverovich. Text Formatting. URL: https://wg21.link/p0645

[P1361]

Victor Zverovich; Daniela Engert; Howard E. Hinnant. Integration of chrono with text formatting. URL: https://wg21.link/p1361